国立大学法人
電気通信大学
The University of Electro-Communications

Pham Laboratory
Integrated circuit design laboratory

「Course」
EDHOC on a Rocket RV64 SoC

「Lesson 2」 Software environment and tests

Khai-Duy Nguyen

2026/12

Today's lesson purpose:

- Install the bare-metal RISC-V toolchain and find your way around the firmware tree (*fpga/sw/export*).
- Understand how one image is built: compiler flags, linker script, startup code, stack check.
- Configure your own two boards, run a first program end to end, then check the peripherals and every crypto primitive on the board.

Prerequisite: Lesson 1 (both boards programmed with the EDHOC bitstream)

Outline

1. Toolchain & Repository
2. Your Own Boards
3. Build System
4. Board Tools
5. Peripheral Tests
6. Crypto Tests
7. Practice

Outline

1. **Toolchain & Repository**
2. Your Own Boards
3. Build System
4. Board Tools
5. Peripheral Tests
6. Crypto Tests
7. Practice

1. Toolchain & Repository (1/6) The RISC-V GCC toolchain

```
git clone https://github.com/riscv-collab/riscv-gnu-toolchain
cd riscv-gnu-toolchain
./configure --prefix=$HOME/opt/riscv --with-multilib-generator="rv64imac-lp64--"
make -j$(nproc)                          # newlib (bare-metal) toolchain
export PATH=$HOME/opt/riscv/bin:$PATH
riscv64-unknown-elf-gcc --version        # ours: 15.1.0
```

- Bare metal: no operating system, no C library at run time — the firmware brings its own *memcpy*, *printf* (kprintf) and startup code.
- Target: **rv64imac / lp64**, the small Rocket of Lesson 1 (no floating point).
- Toolchain somewhere else? Pass the prefix instead of PATH: `make CROSS_COMPILE=/opt/riscv/bin/riscv64-unknown-elf-`
- Also needed: OpenOCD (JTAG), Vivado (programming), Python 3 (host tests).

1. Toolchain & Repository (2/6) The firmware tree: fpga/sw/export/

Directory	What it is
edhoc-wolfssl/	EDHOC RFC 9528, methods 0–3, all 9 cipher suites (Lesson 3)
edhoc-authkem/	post-quantum EDHOC AuthKEM, suites TBD1–TBD3 (Lesson 4)
tests/	peripheral tests, crypto known-answer tests, SD card preparation (sd_prep)
common/	shared by all three: startup, linker, drivers, CBOR, EDHOC core, crypto, radio link, SD/FAT32, board scripts
tools/	mksd.sh — prepares a microSD card on a PC
docs/slides/	these lessons, PPTX and PDF
deviations.txt	the questions asked during the project and how each was answered (dated log)

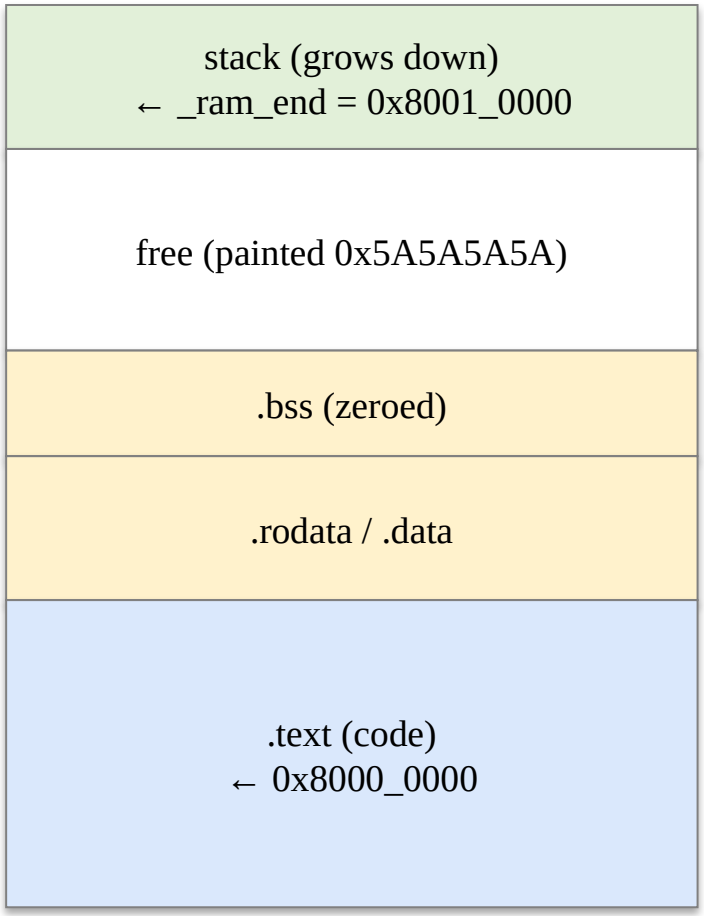
- Everything is vendored: no submodules, nothing to download at build time.
- The packages share *common/*, so copy *export/* as a whole.

1. Toolchain & Repository (3/6) What is in common/

Folder	Contents
src/, include/	crt0.S startup, kprintf, libc stubs, UART/radio transport, MRF24J40 driver, SD card, FAT32, application channel, timing
linker/	main.lds + memory.lds: one image in the 64 KiB scratchpad
mk/	rv64.mk (toolchain, flags), rules.mk (compile / link rules)
edhoc/, cbor/	EDHOC core shared by both packages, deterministic CBOR
crypto/	SHA-256/384, HMAC/HKDF, AES-CCM/GCM, KMAC — our own, small
externals/	Monocypher (ChaCha20, Ed25519), curve25519-donna, mlkem-native
board/	run_one.sh, load_both.sh, program_two.tcl, sd_boot.sh, c232hm.cfg, boards.env, perf_table.py

One copy of each piece of code: both EDHOC packages and the tests link the same files.

1. Toolchain & Repository (4/6) Memory layout of one image



- The linker script places code, data and zeroed data from the bottom of the scratchpad; the stack grows down from the top.
- Each build reserves a stack size (`STACK_SIZE`, per suite and role). The link **fails** if image + reserve do not fit:

```
common/linker/main.lds
ASSERT((_end <= (_ram_end - __stack_reserve)),
  "ERROR: Program + stack exceeds RAM. Reduce the image or lower STACK_SIZE.")
```

- The linker checks that the program **and the configured stack reserve** fit. Whether the real stack stays inside that reserve is measured on the board (next slides).

1. Toolchain & Repository (5/6) Will my build fit? Check it yourself

```
Run this
cd fpga/sw/export/edhoc-wolfssl
make edhoc_m3_initiator CRYPTO_SUITE=4
riscv64-unknown-elf-size build/edhoc_m3_initiator.elf
grep -m1 __stack_size build/edhoc_m3_initiator.map
```

```
Example output – this revision
 text  data   bss    dec   hex filename
25425     0  1640  27065  69b9 build/edhoc_m3_...
... __stack_size = 0x1500        (= 5,376 B)
```

Build (initiator)	text	bss	image	stack reserve	total	fits
suite 4, method 3	25,425	1,640	27,065	5,376	32,441	32 KiB (327 B left)
suite 25, method 0	46,359	1,752	48,111	12,544	60,655	64 KiB only

- **text** = code + constants, **data** = initialised variables, **bss** = zeroed variables (in RAM, not in the file). The ELF file (37,024 B here) also holds symbols; the *.bin* (25,432 B) is what gets loaded.
- Fits = image + reserve ≤ RAM (the linker check) **and** the board's STACK peak ≤ reserve (next slide). The reserve per suite and role is in *mk/config.mk*.
- Bigger curves and signatures cost both code and stack: that is why suite and method decide whether 32 KiB is enough (Lesson 1).

1. Toolchain & Repository (6/6) Startup code and the stack check

```
common/src/crt0.S (outline)
_prog_start:
  la sp, _sp                 // stack at the top of the scratchpad
  // clear .bss
  // paint all free RAM [_end, _ram_end) with 0x5A5A5A5A
  call main
```

```
end of every EDHOC run
STACK peak 5376 B, reserve 8448 B, free 25856 B, headroom 20480
```

- At the end of a run the firmware looks for the first word that is no longer 0x5A5A5A5A: that is the deepest the stack ever went (*stack_highwater()*).
- **peak** = measured use; **reserve** = what the build set aside; **headroom** = how far the stack could still grow into free RAM.
- A run whose peak is over its reserve prints *OVERRAN-RESERVE* and *run_pair.sh* fails it. The reserves in this course are measured peak + 25 %.

Outline

1. Toolchain & Repository
2. **Your Own Boards**
3. Build System
4. Board Tools
5. Peripheral Tests
6. Crypto Tests
7. Practice

2. Your Own Boards (1/3) Find your four serial numbers

```
Run this
cd fpga/sw/export
./common/board/list_boards.sh
```

```
Example output — our lab, yours differ
C232HM JTAG   FTA6GPSN  (C232HM-DDHSL-0)
Arty console  210319B4A439  /dev/ttyUSB4
C232HM JTAG   FTA6JVAK  (C232HM-DDHSL-0)
Arty console  210319B269F6  /dev/ttyUSB1
```

boards.env key	What it is	Which line
INITIATOR_CONSOLE	board A's own USB serial (its console)	Arty console
RESPONDER_CONSOLE	board B's own USB serial	Arty console
INITIATOR_SERIAL	C232HM cable on board A's PMOD JD	C232HM JTAG
RESPONDER_SERIAL	C232HM cable on board B's PMOD JD	C232HM JTAG

- Which is which? Unplug board B's two USB plugs, run the script again: what is left belongs to board A.
- Roles: board A = **initiator** (sends SEND.BIN), board B = **responder** (writes RECV.BIN). Label the boards.

2. Your Own Boards (2/3) Write them into boards.env and check

```
Edit — fpga/sw/export/common/board/boards.env (our values shown)
INITIATOR_SERIAL=FTA6JVAK          # C232HM on board A   <- yours
RESPONDER_SERIAL=FTA6GPSN          # C232HM on board B   <- yours
INITIATOR_CONSOLE=210319B269F6     # Arty A              <- yours
RESPONDER_CONSOLE=210319B4A439     # Arty B              <- yours
```

```
Run this
cd fpga/sw/export/tests && make gpio_test
BOARD=initiator ../common/board/run_one.sh build/gpio_test.elf 5
BOARD=responder ../common/board/run_one.sh build/gpio_test.elf 5
```

You see	Meaning / fix
LEDs walk on board A, then B; console text each time	all four serials right
LEDs walk on the other board	the two cable serials are swapped
LEDs walk, console prints nothing	console serials swapped or wrong
“console … not found”	a console serial is not plugged in or mistyped
OpenOCD error, no LEDs	cable serial wrong, JD wiring, or FPGA not programmed (“dtmcontrol is 0”: Lesson 1, section 6)

2. Your Own Boards (3/3) Lab: your first program, end to end

Objective	load one program into board A and read its result
Starting state	both FPGAs programmed (Lesson 1); boards.env filled in; toolchain and OpenOCD installed

```
Run this
cd fpga/sw/export/tests
make gpio_test
BOARD=initiator ../common/board/run_one.sh build/gpio_test.elf 5
```

```
Example output — board A (shortened)
=== GPIO test (0x64002000, 8 pins) ===
  output_en   : got=ff want=ff PASS
  all high    : got=ff want=ff PASS
  walking ones (watch the LEDs):
    walk: PASS
GPIO TEST: PASS (0 failures)
```

- **What it proves:** JTAG loading works (the program runs at all), UART0 and the console path work (you read it), the GPIO block answers on the bus (read-back PASS), and the pins reach the LEDs (you see them walk).
- **What it does not prove:** anything about SPI, the radio or the SD card — those have their own tests (section 5).
- **If it fails:** nothing printed → (2/3) table; a FAIL line → that register or pin; LEDs dark but PASS → the pins of the bitstream (Lesson 1, section 4).

Outline

1. Toolchain & Repository
2. Your Own Boards
3. **Build System**
4. Board Tools
5. Peripheral Tests
6. Crypto Tests
7. Practice

3. Build System (1/4) How a package Makefile is put together




```
Run this
cd fpga/sw/export/edhoc-wolfssl
make edhoc_m3_initiator CRYPTO_SUITE=2          # -> build/edhoc_m3_initiator.elf
make edhoc_m3_responder CRYPTO_SUITE=2          # same build/ folder, no 'make clean'

cd ../edhoc-authkem
make edhoc_authkem_initiator SUITE=TBD3         # -> build-tbd3/edhoc_authkem_initiator.elf
```

- Blue = shared by both packages (*common/mk*), yellow = per package.
- Output per target: *.elf* (load over JTAG), *.bin* (SD boot), *.map* (who uses how many bytes).

3. Build System (2/4) Compiler and linker flags

```
common/mk/rv64.mk (excerpt)
ARCH_FLAGS = -march=rv64imac_zicsr_zifencei -mabi=lp64 -mcmodel=medany

LDFLAGS  = $(ARCH_FLAGS) -flto -T $(COMMON)/linker/main.lds
LDFLAGS += -nostartfiles -nostdlib -nodefaultlibs
LDFLAGS += -Wl,--defsym=__stack_size=$(STACK_SIZE)
LDFLAGS += -Wl,--gc-sections -Wl,-Map=$(BUILD_DIR)/$(TARGET).map
```

Flag	Why
-mcmodel=medany	code and data at 0x8000_0000 (not near address 0)
-Os, -flto, --gc-sections	optimise for size, link-time optimisation, drop every unused function → small images
-nostdlib -nostartfiles	no C library, our own crt0.S
--defsym=__stack_size	stack reserve for the ASSERT in the linker script
-Map=…map	where every byte went — first stop when an image grows

3. Build System (3/4) A real build bug: stale objects

- All targets of a package share one *build/* folder and differ only in flags (method, role, suite).
- Make only looks at file times. Building *m1_responder* and then *m2_initiator* reused objects compiled for method 1 → **an image of the wrong method**, no error, wrong handshake.
- Fix: an empty stamp file named after a hash of the flags. New flags → new stamp name → every object is older than its stamp → rebuild.
- Check it: `ls build/.cflags-*` before and after changing CRYPTO_SUITE — the name changes and every object compiles again.

```
common/mk/rules.mk
CFLAGS_STAMP = $(BUILD_DIR)/.cflags-$(shell echo '$(CFLAGS) $(INCLUDES)' | md5sum | cut -c1-12)

$(CFLAGS_STAMP): | $(BUILD_DIR)
        @rm -f $(BUILD_DIR)/.cflags-*
        @touch $@

$(BUILD_DIR)/%.o: %.c $(CFLAGS_STAMP)
        @$(CC) -c $(CFLAGS) $(INCLUDES) $< -o $@
```

3. Build System (4/4) Build options

Option	Values	Effect
CRYPTO_SUITE= (wolfssl)	0 1 2 3 4 5 6 24 25	EDHOC cipher suite (default 2)
SUITE= (authkem)	TBD1 TBD2 TBD3	post-quantum suite (default TBD2)
TRANSPORT=	rf | uart	radio on PMOD JC (default) or UART1 wire on JB
DEBUG=1		step-by-step prints, incl. PRK_out
TIMING=1		time of every message step (Lessons 3, 4)
APP_FILE_BYTES=n	bytes	stream n test bytes after the handshake
EXTRA_CFLAGS=-DAPP_SD_XFER		send SEND.BIN card to card (Lesson 5)
PROFILE=production	+ PROVISION_SRC	refuse demo keys, require a real random source

Full lists: *edhoc-wolfssl/README.md*, *edhoc-authkem/README.md*.

Outline

1. Toolchain & Repository
2. Your Own Boards
3. Build System
4. **Board Tools**
5. Peripheral Tests
6. Crypto Tests
7. Practice

4. Board Tools (1/4) Scripts in common/board/

Script	**Does**
program_two.tcl	Vivado batch: program both FPGAs with one bitstream
run_one.sh <elf> [s]	load one board over JTAG, run, print its console
load_both.sh <A.elf> <B.elf>	load responder, then initiator (handshake runs)
boards.env	FTDI serials of the two C232HM cables
c232hm.cfg	OpenOCD config: cable + RISC-V debug tap
sd_boot.sh	write a firmware .bin to the SD boot partition
perf_table.py <logdir>	TIMING=1 logs → one table of message times

- Plus *tools/mksd.sh* (card on a PC) and *tests/sd_prep* (card from the board, Lesson 5).

4. Board Tools (2/4) What happens when you load a program




```
common/board/run_one.sh (core)
stty -F $TTY 57600 raw -echo clocal -crtscts
cat $TTY > $LOG &                                   # start listening first
openocd -c "adapter serial $SERIAL" -f c232hm.cfg -c "poll off" -c "halt" \
  -c "reg mstatus 0x0" -c "reg mie 0x0" -c "reg mtvec 0x80000000" \
  -c "load_image $ELF" -c "resume 0x80000000" -c "shutdown"
sleep "$SECS"; tr -d '\r' < "$LOG"
```

- Interrupts off and the trap vector reset before loading: a previous program cannot fire into the new one.
- The console is opened **before** the program starts, so no early line is lost.

4. Board Tools (3/4) Reading the console

```
initiator, suite 2, method 3
EDHOC OK!
PERF handshake_total   206069790 cyc 2.747597 s
PERF handshake_link    103475867 cyc 1.379678 s
PERF handshake_compute 102593923 cyc 1.367918 s
STACK peak 5376 B, reserve 8448 B, free 25856 B, headroom 20480
PERF encrypt      54759 cyc 0.000730 s 60 bytes 82191 B/s
APP DATA OK
```

- **PERF** lines: *label*, cycles, seconds — and for transfers bytes and B/s. Seconds = cycles / 75 MHz.
- *total* = whole handshake; *link* = waiting for the radio; *compute* = the rest.
- **STACK**: the stack check from section 1.
- **OK / PASS / FAIL** lines are what the sweep scripts look for.

Fixed line shapes make every run machine-readable: *perf_table.py* builds the tables of Lessons 3–5 from saved logs.

4. Board Tools (4/4) Common problems

Symptom	Cause	Fix
OpenOCD: “dtmcontrol is 0”	FPGA not running the EDHOC bitstream (power-off)	program again (program_two.tcl)
No console text	console serial wrong in boards.env	list_boards.sh, then fix boards.env
LEDs react on the other board	cable serials swapped	swap them in boards.env
LIBUSB_ERROR_BUSY	Vivado GUI holds the JTAG port	close the hardware manager or reuse its server
TRANSPORT=uart run hangs	no wire between the boards	JB1 ↔ JB2 crossed between boards, plus GND
Link: “Program + stack exceeds RAM”	image + reserve > 64 KiB	smaller build or lower STACK_SIZE (check the peak first)
Run ends with OVERRAN-RESERVE	stack peak above its reserve	raise STACK_SIZE

Outline

1. Toolchain & Repository
2. Your Own Boards
3. Build System
4. Board Tools
5. **Peripheral Tests**
6. Crypto Tests
7. Practice

5. Peripheral Tests (1/3) Four programs, one per peripheral

Program	Checks	You should see
gpio_test	output enable, all low/high, walking ones (LEDs), XOR, patterns	PASS
uart_test	UART0 console, UART1 registers, optional loopback JB1–JB2	PASS (loopback SKIP without jumper)
spi_test	SPI0 registers, real SD card answers CMD0 with R1 = 0x01	PASS
jtag_test	loaded over JTAG at all; reads mhartid, marchid, misa	PASS

```
Run this
cd fpga/sw/export/tests
make periph                                          # all four
BOARD=initiator ../common/board/run_one.sh build/spi_test.elf 3
```

Console output of each: Lesson 1, section 5.

5. Peripheral Tests (2/3) How a test is written

tests/src/periph_gpio.c (simplified)

```
/* 2. walking ones — proves each pin is independently addressable */
kprintf("  walking ones (watch the LEDs):\r\n");
for (int i = 0; i < GPIO_WIDTH; i++) {
    GPIO_REG(GPIO_OUTPUT_VAL) = 1u << i;
    delay();
    if ((GPIO_REG(GPIO_OUTPUT_VAL) & 0xff) != (1u << i))
        fails++;
}
kprintf("    walk: %s\r\n", fails ? "FAIL" : "PASS");
```

- Write a register, read it back, print **PASS / FAIL** per phase and one final verdict.
- Something you can see: the LEDs walk.
- A test that needs outside help (UART loopback, SD card) prints **SKIP** instead of FAIL when the help is missing.
- **SKIP is not PASS**: that check did not run. The final PASS covers only the checks that ran — read every line, not just the last one.

5. Peripheral Tests (3/3) What the SD test taught us

```
raw bytes after each command, initiator board
CMD0: 3f 1 ff ff ff ff ...      <- first CMD0 after power-up
CMD0: ff 1 ff ff ff ff ...      <- second CMD0
CMD8: ff 1 0 0 1 aa ff ...      <- voltage check echoes 0x1AA
```

- The first version reported SD **FAIL** on both boards (R1 = 0x3f / 0x7f). The cards were fine.
- Right after power-up the card sends one junk byte with bit 7 clear before its real answer; the test took the junk for the answer.
- Fix: send CMD0 up to 10 times until R1 = 0x01 (tests and firmware). Most SD drivers do the same.
- Lesson: print the **raw bytes** before blaming the hardware.

Outline

1. Toolchain & Repository
2. Your Own Boards
3. Build System
4. Board Tools
5. Peripheral Tests
6. **Crypto Tests**
7. Practice

6. Crypto Tests (1/5) What is tested, against which vectors

Primitive	Implementation	Test vectors
SHA-256 / SHA-384	common/crypto	FIPS 180-4
HMAC-SHA-256	common/crypto	RFC 4231, test case 1
HKDF-SHA-256	common/crypto	RFC 5869, test case 1
AES-128-CCM	common/crypto	RFC 3610
AES-128-GCM / AES-256-GCM	common/crypto	NIST GCM spec, test cases 2 / 14
X25519	curve25519-donna	RFC 7748 §6.1
ML-KEM-512 / 768 / 1024	mlkem-native (FIPS 203)	FIPS 203 known answers + rejection

- The tests link the **same files** the EDHOC firmware uses — not a separate copy — so a PASS here covers the code that runs in the handshake.

6. Crypto Tests (2/5) crypto_test on the board

```
tests/build/crypto_test.elf
=== Classical crypto KAT (common/crypto, donna) ===
  vectors: FIPS 180-4, RFC 4231, RFC 5869, RFC 3610, NIST GCM, RFC 7748
  SHA-256       : PASS
  SHA-384       : PASS
  HMAC-SHA-256  : PASS
  HKDF-SHA-256  : PASS
  AES-128-CCM   : PASS
    CCM tag     : PASS
    CCM decrypt : PASS
  AES-128-GCM   : PASS
    GCM tag     : PASS
    GCM decrypt : PASS
  AES-256-GCM   : PASS
    GCM tag     : PASS
    GCM decrypt : PASS
  X25519 shared : 8917229 cycles
  X25519        : PASS
CRYPTO TEST: PASS (0 failures)
```

- AEAD tests check three things: ciphertext, tag, and that decrypting the expected ciphertext verifies.
- On a mismatch the test prints *got* and *want*, so the console alone is enough to debug.
- X25519 key agreement: 8.9 M cycles = 0.12 s at 75 MHz.

6. Crypto Tests (3/5) mlkem_test on the board

```
tests/build/mlkem_test.elf
=== ML-KEM (FIPS 203, mlkem-native) test  levels 512/768/1024 ===
-- ML-KEM-512 (pk=800 sk=1632 ct=768) --
  keypair KAT : PASS
  encaps KAT  : PASS
  decaps      : PASS
  reject KAT  : PASS
-- ML-KEM-768 (pk=1184 sk=2400 ct=1088) --
  keypair KAT : PASS   encaps KAT : PASS   decaps : PASS   reject KAT : PASS
-- ML-KEM-1024 (pk=1568 sk=3168 ct=1568) --
  keypair KAT : PASS   encaps KAT : PASS   decaps : PASS   reject KAT : PASS
MLKEM TEST: PASS (0 failures)
```

- Three security levels in one image: the library is compiled three times with different prefixes (*mlk512_*, *mlk768_*, *mlk1024_*).
- **reject KAT**: a corrupted ciphertext must give the defined pseudo-random key, not an error — FIPS 203 implicit rejection.
- Needs a 16 KiB stack (ML-KEM-768 decapsulation is the deepest call).

6. Crypto Tests (4/5) How a known-answer test is written

```
tests/src/crypto_classic.c (shortened)
static int test_x25519(void)
{
    /* RFC 7748 section 6.1: Alice and Bob must agree on K */
    static const uint8_t a_priv[32] = { 0x77,0x07,0x6D,0x0A, ... };
    static const uint8_t b_pub[32]  = { 0xDE,0x9E,0xDB,0x7D, ... };
    static const uint8_t want[32]   = { 0x4A,0x5D,0x9D,0x5B, ... };
    uint8_t got[32];

    curve25519_donna(got, a_priv, b_pub);
    return report("X25519        ", got, want, 32);   // PASS, or got/want dump
}
```

- Inputs and expected output copied **byte for byte** from the standard.
- One function per primitive, one *report()* that compares and prints.
- Add a primitive: one more function and one more line in *main()*.

6. Crypto Tests (5/5) Tests on the PC

Host test	What it runs	You should see
make loopback-test (wolfssl)	initiator + responder as two threads, one suite × method per run, PRK_out compared	PRK_out match -> PASS
make unit-test, loopback-test (authkem)	encodings, KEM, error messages; full 5-message handshake, per suite	PASS
make rfc9529-test	every byte of the RFC 9529 example traces (Lesson 3)	PASS for suites 0, 2, 24
make fat32-host-test (tests/)	FAT32 code on a real disk image, checked with fsck.fat	every check PASS

- Same C files as the firmware, compiled for the PC: seconds per run, a debugger, no boards needed.
- Rule used throughout: **host test first, then the board**.

Outline

1. Toolchain & Repository
2. Your Own Boards
3. Build System
4. Board Tools
5. Peripheral Tests
6. Crypto Tests
7. **Practice**

7. Practice (1/3) Exercises

Exercise 1:

- Build the toolchain, then `make periph crypto` in *tests/*. Note each image size from the *size* output.

Exercise 2:

- Run all six test programs on both boards and collect the console output.

Exercise 3:

- Add a SHA-256 test for the empty string (FIPS 180-4 example) to *crypto_classic.c* and make it PASS on the board.
- Then break one byte of the expected value: what does the console show?

7. Practice (2/3) What to put in the report

- Exercise 1: toolchain version, the build commands, image sizes.
- Exercise 2: the six console logs per board; explain any SKIP.
- Exercise 3: your test function, the PASS output and the FAIL output with the broken byte.

7. Practice (3/3) Answer notes

Exercise	What you should find	Why
1. Sizes	each image's text + bss from riscv64-unknown-elf-size; crypto_test and mlkem_test are the largest test images	they link the most crypto code
2. Six tests	every final line PASS on both boards; uart_test shows UART1 loop: SKIP without the jumper	SKIP = not run; nothing failed
3. SHA-256(")	want = e3b0c442 98fc1c14 9afbf4c8 996fb924 27ae41e4 649b934c a495991b 7852b855 → PASS	FIPS 180-4 example
3. broken byte	“SHA-256 …: FAIL”, then got and want, differing in that byte; final line CRYPTO TEST: FAIL (1 failures)	the test compares every byte and counts failures

- Restore after exercise 3: undo the broken byte (or, from *fpga/sw/export*, `git checkout tests/src/crypto_classic.c`), rebuild, run: PASS again.

Today's lesson highlight:

- Bare-metal RV64IMAC firmware: own startup, own libc pieces, one image in the 64 KiB scratchpad; the linker refuses image + stack reserve that do not fit, the STACK line shows the real peak.
- One shared *common/* tree; makefiles split into shared flags/rules and per-package targets; a flags stamp prevents stale objects.
- Load over JTAG with *run_one.sh*; PERF / STACK / PASS lines make every run machine-readable.
- You can now: set up boards.env for your boards, check whether a build fits and how much stack margin it has, load either board, and tell PASS, FAIL and SKIP apart in peripheral and crypto test output.

Next lesson:

「 Lesson 3 」 EDHOC (RFC 9528) on two boards

THANK YOU

2026/12